Conditional Reception

A ractor can be made to be picky about what it lets in the front door by using the Ractor.receive_if method, which takes a block argument. If another ractor tries to send to a ractor that is waiting on receive_if, the receiving ractor will call the block argument on the objects sent. If the block returns a truthy value, then recieve_if returns the object just the same way that plain ordinary recieve does. If the block returns a falsey value, the ractor continues to wait. But the failing object stays at the head of the line, so if the ractor ever does get an object that passes the block, all the failed objects are still in the entrance queue and are able to get picked up by future receive calls in the ractor.

Waiting on Multiple Ractors

If you have multiple ractors you might be waiting on, and you want to respond to whichever of them yields a value on the outgoing port first , you can use Ractor.select. The argument to Ractor.select is an arbitrary number of ractors, as in Ractor.select(r1, r2, r3). The value returned is a two object array, the first of which is the ractor that has put the value on the port, and the second is the value itself, so r, val = Ractor.select(r1, r2, r3).

Now, there are a couple of wierdnesses here. First off, one of the Ractors in the argument could be the Ractor making the call, as in Ractor.select(r1, Ractor.current). If the current ractor is somehow the one that emits the value, then the select call still returns the value, but instead of returning the ractor, it returns :receive.

Also, you can use Ractor.select to deal with multiple other ractors that you expect to take rather than yield, in which case, our call needs to provide the value for the take call, which you do with a yield_value keyword argument: Ractor.select(r1, r2, yield_value: 37). In the yield case, the return values are :yield and nil. It's frankly not clear why those aren't two different methods.

You can slam shut either the entrance or exit doors with the close_incoming and close_outgoing methods. Attempts to access a closed port on a ractor return an exception, as do attempts to access the outgoing port of a ractor that has ended and returned its last value.

What's Next

That covers the basics of threading in Ruby. We've talked about basic threads, using system processes, fibers, and ractors. Now let's look at how we can use testing to help ensure that our code does what we expect.